Floe — Interpreter and AOT Compiler

Course: Compiler Construction · **Submitted to:** Sir Faisal Aslam

Team: Hamza Nazakat (23L-0952) · Muhammad Ahmed (23L-0733) · Agha Ali Hassan (23L-0637)

Floe is a small, statically typed **data-pipeline language**. Data flows left to right through transformation stages joined by the `|>` operator:

```
let nums = [1, 2, 3, 4, 5];

nums
    |> filter(x -> x > 3)
    |> map(x -> x * 2)
    |> print();            // [8, 10]
```

This project contains both deliverables:

Phase	Program	What it is	Built with
Phase 1 (60%)	`build/floe`	**Interpreter.** The Bison grammar actions execute each statement as soon as it is parsed. No intermediate code is generated.	Flex + Bison + C++
Phase 2 (40%)	`build/floec`	**Ahead-of-time compiler.** The same Bison grammar actions generate **LLVM IR**, which is compiled into a native Linux executable.	Flex + Bison + C++ + LLVM 16

Both phases share the lexer, the parser and the semantic analyzer, and they print **identical output** for every test program.

---

1. Installing the dependencies (Linux)

Tested on **Ubuntu 24.04** (also in WSL 2) with Flex 2.6.4, Bison 3.8.2, GCC 13 and LLVM/Clang 16.0.6.

```
sudo apt update
sudo apt install -y build-essential flex bison     # Phase 1 (interpreter)
sudo apt install -y llvm-16 llvm-16-dev clang-16   # Phase 2 (compiler)
```

Check the tools:

```
flex --version && bison --version && g++ --version && llvm-config-16 --version
```

Notes:

- `llvm-16-dev` provides the headers and `libLLVM-16.so` that `floec` links against. The Makefile finds `llvm-config-16` (or `llvm-config`) automatically; you can override it with `make`

`LLVM_CONFIG=/path/to/llvm-config`. Other LLVM versions from 16 upward should work but were not tested.

- `clang-16` is optional. It is needed only for compiling the generated `.ll` file by hand (section 4.3). `floec` itself only needs `cc`.
- On Windows, use WSL 2 with Ubuntu and run every command inside it.

2. Compiling the project

From the project directory:

```
make                # builds both programs (= make interpreter compiler)
make interpreter    # Phase 1 only: build/floe  (does not need LLVM)
make compiler       # Phase 2 only: build/floec + build/floe_runtime.o
make test           # runs the whole test suite (should end "... passed, 0 failed")
make clean          # removes build/ and test output
```

The build generates `build/parser.cpp` / `parser.hpp` with Bison and `build/lexer.cpp` with Flex, compiles everything into `build/*.o`, and links:

- `build/floe` — the interpreter;
- `build/floec` — the compiler;
- `build/floe_runtime.o` — the C runtime that every compiled program is linked with.

3. Running programs — Phase 1: the interpreter

```
./build/floe program.floe            # run a program
```

Example:

```
$ ./build/floe tests/mergesort.floe
[1, 2, 3, 5, 8, 9]
```

Other modes of `floe`, useful to see each compiler phase separately:

Command	Shows
`./build/floe --tokens file.floe`	the token stream produced by Flex
`./build/floe --parse file.floe`	whether the program is syntactically valid
`./build/floe --ast file.floe`	the abstract syntax tree built by Bison
`./build/floe --check file.floe`	syntax + type checking, without running
`./build/floe --typed-ast file.floe`	the AST after semantic analysis, with types
`./build/floe file.floe` (or `--run`)	runs the program

Exit status: `0` success, `1` lexical/syntax/semantic/runtime error, `2` bad usage or unreadable file.

4. Running programs — Phase 2: the AOT compiler

4.1 Compile to an executable

```
./build/floec program.floe -o program     # Floe -> LLVM IR -> object -> executable
./program
```

```
$ ./build/floec tests/mergesort.floe -o mergesort
$ ./mergesort
[1, 2, 3, 5, 8, 9]
```

Without `-o` the output is named after the input (`tests/mergesort.floe` → `tests/mergesort`).

4.2 Options

Option	Effect
`-o <path>`	output file name
`--emit-llvm`	write the LLVM IR as text (`.ll`) instead of an executable
`-c`	write a native object file (`.o`) instead of an executable
`-O`	optimise with LLVM's standard `-O2` pipeline

Environment variables: `CC` chooses the linker driver (default `cc`); `FLOE_RUNTIME` gives the path of `floe_runtime.o` if it is not next to `floec`.

4.3 Looking at, and compiling, the LLVM IR by hand

```
./build/floec --emit-llvm tests/basic_pipeline.floe -o basic.ll
cat basic.ll                                                    # human-readable LLVM IR
llc-16 -relocation-model=pic -filetype=obj basic.ll -o basic.o # IR -> object (LLVM backend)
cc basic.o build/floe_runtime.o -o basic -lm                    # link with the runtime
./basic                                                         # prints [8, 10]
```

5. Using the language

5.1 A complete example

Save this as `scores.floe`:

```
// Student scores: functions, a named pipe, pipelines, loops and if/else.
pipe passing = filter(s -> s >= 50);

fn grade(score: int) -> string {
    if score >= 80 { "A" } else if score >= 65 { "B" } else if score >= 50 { "C" } else { "F" }
}

fn average(values: int[]) -> float {
    to_float(sum(values)) / to_float(len(values))
}

let scores = [72, 45, 91, 66, 38, 85];

print("sorted:", scores |> sort());
print("passing:", scores |> passing);
print("grades:", scores |> map(s -> grade(s)));
print("average:", average(scores));

let best = 0;
for s in scores {
    if s > best { best = s; }
}
print("best:", best, "=", grade(best));
```

Run it with either phase:

```
$ ./build/floe scores.floe                      # Phase 1: interpret
sorted: [38, 45, 66, 72, 85, 91]
passing: [72, 91, 66, 85]
grades: ["B", "F", "A", "B", "F", "A"]
average: 66.1666666666667
best: 91 = A

$ ./build/floec scores.floe -o scores && ./scores   # Phase 2: compile, then run
sorted: [38, 45, 66, 72, 85, 91]
passing: [72, 91, 66, 85]
grades: ["B", "F", "A", "B", "F", "A"]
average: 66.1666666666667
best: 91 = A
```

5.2 Language reference

Comments: `// to end of line` and `/* block */`.

Types: `int` (64-bit), `float` (64-bit), `bool`, `string`, and arrays `int[]`, `float[][]`, … An `int` is converted to `float` automatically where a `float` is expected.

Variables:

```
let count = 3;                 // type inferred: int
let ratio: float = 0.75;       // explicit type
let empty: int[] = [];         // an empty array needs its type
count = count + 1;             // assignment
let grid = [[1, 2], [3, 4]];
grid[0][1] = 20;               // array element assignment
```

Operators (highest precedence first): unary `-` `!` · `*` `/` `%` · `+` `-` · `<` `>` `<=` `>=` · `==` `!=` · `&&` · `||` · `|>`. `+` also joins strings. `&&` and `||` short-circuit. Comparisons do not chain: write `a < b && b < c`.

Functions return the value of their last expression (no `return` keyword). Recursion is allowed. A function without `-> type` returns nothing.

```
fn factorial(n: int) -> int {
    if n <= 1 { 1 } else { n * factorial(n - 1) }
}
fn report(label: string, value: int) {
    print(label, value);
}
```

Control flow:

```
if x > 0 { print("positive"); } else if x == 0 { print("zero"); } else { print("negative"); }
let sign = if x > 0 { 1 } else { -1 };      // if as a value (needs an else)
while i < 10 { i = i + 1; }
for v in [3, 1, 4] { print(v); }
```

Arrays and slices: `xs[i]` (bounds-checked), `xs[a..b]` (a copy of elements `a` to `b-1`), `len(xs)`. Arrays are references: after `let b = a;`, changing `b[0]` also changes `a[0]`.

Pipelines: `value |> f(args)` means `f(value, args)`. Any built-in or user function can be a stage; a stage without arguments can omit the `()`.

```
[5, 3, 9, 1] |> sort() |> reverse() |> print();     // [9, 5, 3, 1]
words |> map(w -> len(w)) |> sum() |> print();
```

Lambdas `x -> expression` are the second argument of `filter` and `map`. They may read outer variables but may not assign them.

Named pipes are reusable, composable pipeline fragments:

```
pipe evens = filter(x -> x % 2 == 0);
pipe evens_doubled = evens |> map(x -> x * 2);
[1, 2, 3, 4, 5, 6] |> evens_doubled |> print();    // [4, 8, 12]
```

Built-in functions:

Function	Result
`print(a, b, …)`	prints the values separated by spaces, then a newline
`len(xs)` / `len(s)`	length of an array / a string
`append(xs, x)`	a new array with `x` added at the end
`filter(xs, x -> cond)`	the elements for which `cond` is true
`map(xs, x -> expr)`	`expr` applied to each element
`sort(xs)`	sorted copy (int, float or string elements)
`reverse(xs)`	reversed copy
`sum(xs)`, `min(xs)`, `max(xs)`	of an int[] or float[]
`range(a, b)`	`[a, a+1, …, b-1]`
`to_float(n)`, `to_int(x)`, `to_string(x)`	conversions

Rules to remember: statements end with `;` (blocks don't). A name must be declared before it is used. A block's last expression without `;` is its value.

5.3 Error messages

Every error has a `file:line:column` position. For example:

```
tests/errors/lexer_errors.floe:2:11: lexical error: unexpected character '@'
tests/errors/parser_errors.floe:3:9: syntax error, unexpected ';'
tests/errors/sema_errors.floe:4:14: semantic error: initial value of 'b' should be 'int', but it is
'string'
tests/errors/runtime_index.floe:4:13: runtime error: index 3 is out of bounds for an array of length
3
```

6. Testing

```
make test               # everything (72 checks)
make test-lexer         # token stream of every program; lexer errors detected
make test-parser        # every program parses; syntax errors detected
make test-sema          # every program type-checks; exact semantic error messages
make test-interpreter   # Phase 1: output == tests/expected/<name>.out
make test-compiler      # Phase 2: compile + run, output == tests/expected/<name>.out
```

The last two use the **same** expected files, so they prove both phases behave identically, including runtime errors and exit codes. Expected end of `make test`:

```
PASS  compiler     tests/errors/runtime_slice.floe

72 passed, 0 failed
```

7. Project files

Path	Description
`Makefile`	Builds `floe`, `floec`, the runtime; test targets.
`src/lexer.l`	**Flex** lexer: tokens, positions, lexical errors.
`src/lexer.h`	Lexer interface shared with the parser and drivers.
`src/parser.y`	**Bison** LALR(1) grammar. Builds the AST; its `items` action hands each item to the interpreter or code generator.
`src/ast.h`, `src/ast.cpp`	AST node definitions, AST printer, deep copy.
`src/item_handler.h`	Interface between the grammar action and the two back ends.
`src/sema.h`, `src/sema.cpp`	Semantic analysis: scopes, symbol tables, type checking, pipeline rewriting.
`src/interpreter.h`, `src/interpreter.cpp`	**Phase 1** interpreter.
`src/main.cpp`	The `floe` command (interpreter and front-end modes).
`src/codegen.h`, `src/codegen.cpp`	**Phase 2** LLVM IR generator.
`src/floec.cpp`	The `floec` command (IR → object → executable).
`src/driver.h`, `src/driver.cpp`	Shared "open file and run the parser" code.
`src/runtime/floe_runtime.h`, `.c`	C runtime: arrays, strings, printing, runtime errors.
`tests/*.floe`	Valid example programs (12).
`tests/expected/`	Expected output (`.out`) and error messages (`.err`).
`tests/lexer/edge_cases.floe`	Lexer stress test.
`tests/errors/*.floe`	Programs with lexical, syntax, semantic and runtime errors.
`tests/run_tests.sh`	Test runner used by `make test`.
`docs/lexer.md`	How the lexer was built.
`docs/parser.md`	How the parser and AST were built.
`docs/semantic-analysis.md`	How type checking works.
`docs/interpreter.md`	Phase 1: how the interpreter works, how to check it, its output.
`docs/codegen.md`	Phase 2: LLVM IR generation and AOT compilation.
`floe-compiler-plan.md`	Original project plan, with progress ticked off.
`README.md`, `README.pdf`	This document (the PDF also contains `docs/interpreter.md` and `docs/codegen.md` as appendices).
`docs/make_readme_pdf.py`	Regenerates `README.pdf` from the Markdown (needs Python `markdown` + Chrome/Chromium).

Example programs in `tests/`

Program	Shows
`basic_pipeline.floe`	`filter` + `map` pipeline
`named_pipe.floe`	`pipe` declarations
`if_in_pipeline.floe`	`if` inside a lambda
`functions_and_loops.floe`	functions, recursion, `for`, `bool`
`loops.floe`	`while`, `for`, `&&`
`mergesort.floe`	recursion, slices, `append`
`types_and_literals.floe`	every type, `else if`, nested arrays
`builtins.floe`	every built-in function
`strings_and_floats.floe`	strings, conversions, int → float promotion
`scopes_and_globals.floe`	shadowing, globals in functions, array references
`algorithms.floe`	Fibonacci, GCD, prime sieve, bubble sort, binary search, deep recursion
`scores.floe`	the example program of section 5.1

APPENDIX A

Phase 1 — The Floe Interpreter (Flex + Bison)

Phase 1 turns the front end into a working **interpreter**: `build/floe` reads a `.floe` file and runs it. As the assignment requires, it uses **only Flex and Bison** (plus C/C++). No intermediate code is generated, and LLVM is not needed to build or run it. The program is executed by the C++ code that the **Bison grammar actions** call.

```
source.floe ──► Flex lexer ──► Bison parser ──┐
                                              │  grammar action of `items: items item`
                                              ▼
                              semantic analysis of that item ──► execute it now
```

---

1. What we did

Task	Where
Bison action that hands each finished top-level item to the interpreter	`src/parser.y` (`items` rule, `dispatch_item`)
Back-end interface used by that action	`src/item_handler.h`
Tree-walking evaluator: values, environments, statements, expressions	`src/interpreter.cpp`
All built-in functions (`print`, `len`, `append`, `filter`, `map`, `sort`, …)	`Interpreter::Impl::builtin`
Runtime errors with source positions (index out of bounds, division by zero, …)	`RuntimeError`
Deep recursion support (1 GiB interpreter stack, depth limit 100 000)	`run_with_large_stack` in `src/main.cpp`
Identical print formatting to compiled programs	the interpreter calls `floe_print_*` from `src/runtime/floe_runtime.c`
Command-line modes `--run`, `--check`, `--typed-ast`, `--parse`, `--ast`, `--tokens`	`src/main.cpp`
Golden-output tests	`tests/expected/*.out`, `make test-interpreter`

2. Files involved

File	Role
`src/parser.y`	The `items` action calls `dispatch_item(*item)` right after the item is reduced.
`src/item_handler.h`	`ItemHandler` interface + the global `floe_item_handler` the parser calls.
`src/interpreter.h/.cpp`	The interpreter (an `ItemHandler`).
`src/sema.h/.cpp`	Type checking, run on each item before it executes (see semantic-analysis.md).
`src/driver.h/.cpp`	`parse_file()`: opens the file, installs the handler, calls `yyparse()`.
`src/main.cpp`	The `floe` command.
`src/runtime/floe_runtime.c`	Print and number-format helpers shared with Phase 2.

---

3. How we built it

3.1 Execution is driven by the Bison grammar

The classic Bison calculator prints each result inside a grammar action (`line: exp '\n' { printf("%g\n", $1); }`). Floe works the same way, one level up. In `src/parser.y` every top-level item (a function, a named pipe or a statement) is reduced by this rule:

```
items:
    %empty              { $$ = new floe::NodeList(); }
  | items item
        {
            /* Hand the finished item to the interpreter / code generator. */
            $$ = $1;
            if ($2) {
                $$->emplace_back($2);
                dispatch_item(*$2);
            }
        }
  ;
```

`dispatch_item` calls `floe_item_handler->handle(item)`. For `./build/floe program.floe` the handler is the interpreter, so **each statement runs as soon as the parser has recognised it**, before the rest of the file is parsed:

- `fn …` is type-checked, then remembered so later statements can call it;
- `pipe …` is checked and remembered (semantic analysis inlines it later);
- any other statement is type-checked, then executed at once.

The statement's subtree is needed only because a function body or loop body must run many times; it is never translated into another form. The same hook is used in Phase 2 to emit LLVM IR instead (codegen.md).

Consequences, which are typical of a syntax-directed interpreter:

- If line 13 has a syntax error, lines 1–12 have already run. After the first lexical, syntax or semantic error nothing more is executed. Parsing and checking continue so that all errors are reported.
- Names must be declared before use (as in C), because a statement runs before the later lines have even been read.

3.2 Values

```
using ArrayRef = std::shared_ptr<std::vector<Value>>;
struct Value {
    std::variant<std::monostate, long long, double, bool, std::string, ArrayRef> data;
};
```

`monostate` is "no value" (void). `int` is a 64-bit `long long`, `float` a `double`. **Arrays are references:** `let b = a;` makes `b` and `a` the same array, so `b[0] = 99` is visible through `a`. `append`, slices, `filter`, `map`, `sort` and `reverse` always return **new** arrays. The compiled code in Phase 2 has exactly the same semantics.

3.3 Environments (run-time scopes)

An `Environment` is a map from names to `Value`s plus a pointer to the enclosing environment. Lookups walk outwards. The interpreter creates one:

When	Parent
program start (globals)	—
each block `{ … }` evaluated	the current environment
each `for` iteration	the current environment
each lambda application	the current environment (so lambdas can read outer variables)
each function call	the **global** environment (functions see parameters + globals)

These mirror the compile-time scopes of semantic analysis exactly, so a lookup that sema accepted always succeeds.

3.4 Statements and expressions

`execute()` handles statements and `evaluate()` handles expressions, each with a `switch` on the node kind:

- `let` evaluates and defines; assignment replaces a variable or an array element (with a bounds check);
- `while` re-evaluates its condition each time round; `for x in xs` fixes the length when the loop starts;
- a block's value is its trailing expression, which makes implicit return work: a function returns the value of its body block;
- `&&` and `||` short-circuit;

- integer `+ - *` wrap around on overflow (as in the compiled code) instead of invoking C++ undefined behaviour;
- `if` evaluates only the chosen branch and yields its value.

Because semantic analysis already turned pipelines into nested calls and inlined named pipes, the interpreter has no pipeline code at all. `values |> filter(x -> x > 10) |> print()` arrives as `print(filter(values, x -> x > 10))`.

3.5 Built-ins and lambdas

`filter` and `map` evaluate their source array, then run the lambda body once per element in a fresh environment where the parameter is bound to the element. `sort` copies and uses `std::sort`; `sum`, `min`, `max`, `range`, `append`, `reverse`, `len`, `to_float`, `to_int`, `to_string` are a few lines each. `print` uses the runtime library's `floe_print_int`, `floe_print_float` and so on. So `2.0` prints as `2.0` (not `2`), and `[1, 2]` and `["a", "b"]` are formatted identically by the interpreter and by compiled programs.

3.6 Runtime errors

An error throws `RuntimeError{location, message}`. The interpreter catches it, prints `file:line:col: runtime error: message` to stderr after flushing stdout, stops the program and exits with status 1.

Error	Message
`xs[i]` with `i` outside `0..len-1`	`index 3 is out of bounds for an array of length 3`
`a / 0`, `a % 0` (int)	`division by zero`
`xs[a..b]` outside the array	`slice [2..5] is out of bounds for an array of length 3`
`min`/`max` of `[]`	`min() of an empty array`
recursion deeper than 100 000 calls	`stack overflow: recursion is too deep`

Float division by zero follows IEEE rules (`inf`, `nan`) and is not an error.

---

4. How to check it

```
make interpreter                      # builds build/floe (Flex + Bison + g++)
./build/floe tests/mergesort.floe     # run a program
./build/floe tests/algorithms.floe
./build/floe tests/errors/runtime_index.floe ; echo "exit status $?"
make test-interpreter                 # every program vs. tests/expected/*.out
```

`make test-interpreter` runs every `tests/*.floe` and compares its stdout with `tests/expected/<name>.out`. It also runs every `tests/errors/runtime_*.floe` and checks both the output printed before the error and the exact error message (`.err`), plus exit status 1.

5. Output

```
$ ./build/floe tests/basic_pipeline.floe
[8, 10]

$ ./build/floe tests/mergesort.floe
[1, 2, 3, 5, 8, 9]

$ ./build/floe tests/algorithms.floe
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
12 1
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
[1, 2, 4, 7, 8, 9]
3 -1
10000

$ ./build/floe tests/builtins.floe
5 5
[1, 3, 5, 7, 9]
[7, 1, 9, 3, 5]
25 1 9
...

$ ./build/floe tests/errors/runtime_index.floe ; echo "exit status $?"
30
tests/errors/runtime_index.floe:4:13: runtime error: index 3 is out of bounds for an array of length
3
exit status 1

$ make test-interpreter
PASS  interpreter  tests/algorithms.floe
PASS  interpreter  tests/basic_pipeline.floe
...
PASS  interpreter  tests/errors/runtime_slice.floe

16 passed, 0 failed
```

Phase 2 — The Floe AOT Compiler (LLVM IR)

Phase 2 reuses the same Bison grammar. This time the grammar action hands each item to a **code generator** instead of the interpreter. The code generator emits **LLVM Intermediate Representation**, and LLVM compiles that IR into a native object file, which is linked into an executable. This is an **ahead-of-time (AOT) compiler**: `build/floec`.

```
source.floe ─► Flex ─► Bison ─► (items action) ─► semantic analysis ─► LLVM IRBuilder
                                                                              │
                         executable ◄── cc + floe_runtime.o ◄── object file ◄── LLVM backend (x86-
64)
```

---

1. What we did

Task	Where
LLVM `Module`, `IRBuilder`, native `TargetMachine` set up	`CodeGenerator::Impl` constructor
Bison action → code generator (same hook as Phase 1)	`src/parser.y`, `src/item_handler.h`
Functions → LLVM functions `@fn.<name>` (recursion, `phi` returns)	`emit_function`
Top-level statements → body of `@main`; top-level `let` → global `@g.<name>`	`handle`, `emit_statement`
`let`/variables → `alloca` + `store`/`load` (promoted to registers by `-O`)	`create_local`, `define_local`
Arithmetic, comparisons → `add`/`sub`/`mul`/`sdiv`/`srem`, `fadd` …, `icmp`/`fcmp`	`emit_binary`
Division-by-zero check, `INT_MIN / -1` safe	`emit_int_division`
`&&`/`||` short circuit, `if`/`else` → basic blocks + `phi`	`emit_short_circuit`, `emit_if`
`while`, `for-in` → loop blocks with a back edge	`emit_statement`, `emit_counted_loop`
`filter`/`map` → **inline** loops; the lambda body is emitted inside the loop	`emit_lambda_loop`
Arrays, slices, strings, sorting, printing → C runtime library	`src/runtime/floe_runtime.c`
`verifyFunction` / `verifyModule` on everything generated	`emit_function`, `finish`
Optional LLVM `-O2` pipeline	`optimize`
Write `.ll`, write `.o`, link the executable	`write_ir`, `write_object`, `src/floec.cpp`
Tests: every program compiled and run, output compared with Phase 1's	`make test-compiler`

2. Files involved

File	Role
`src/codegen.h`	`CodeGenerator` interface (LLVM headers hidden behind a pimpl).
`src/codegen.cpp`	IR generation with the LLVM 16 C++ API.
`src/floec.cpp`	The `floec` command: parse → IR → object → link.
`src/runtime/floe_runtime.h/.c`	C runtime linked into every compiled program.
`build/floe_runtime.o`	Built by `make compiler`; `floec` finds it next to itself.

---

3. How we built it

3.1 Driving code generation from Bison

`floec` installs a `CodeGenerator` as `floe_item_handler` and calls `yyparse()`. When the `items` action reduces an item, `CodeGenerator::handle` type-checks it, then:

- **function** → a new LLVM function `define internal … @fn.name(…)`. The builder's insertion point is saved, the function is emitted, and then the insertion point returns to `@main`;
- **top-level statement** → instructions appended to `@main`, which was created up front and is closed with `ret i32 0` after parsing;
- **top-level `let`** → an internal global `@g.name`, so functions declared later can read and write it;
- **named pipe** → nothing (semantic analysis inlines it).

The `fn.` and `g.` prefixes keep Floe names from clashing with C symbols such as `main` or `printf`.

3.2 Types and values

Floe	LLVM
`int`	`i64`
`float`	`double`
`bool`	`i1`
`string`	`ptr` (to a NUL-terminated string)
`T[]`	`ptr` (to a runtime `FloeArray { i64 length; i64 capacity; i64* data }`)

Array elements live in uniform **8-byte slots**. Before a store, a value is `encode`d (`bitcast double→i64`, `zext i1→i64`, `ptrtoint`); after a load it is `decode`d. So one runtime array type serves every element type, nested arrays included.

Local variables are `alloca`s placed in the function's entry block, the form LLVM's `mem2reg` pass turns into SSA registers when `-O` is used.

3.3 Control flow

- **if/else**: `br i1 %cond, label %if.then, label %if.else`. Each branch ends with `br label %if.end`. When the `if` is used as a value, a `phi` node in `if.end` picks the branch result. `else if` chains nest.
- **while**: `while.cond` → `while.body` → back edge to `while.cond`, and exit to `while.end`.
- **for x in xs**, **filter**, **map**: a counted loop (`cond`, `body`, `next`, `end` blocks) over `0 … len-1`.
- **&& / ||**: the right operand is in its own block. A `phi` merges the short-circuit constant with the right operand's value.
- **a / b, a % b**: `icmp eq b, 0` branches to a block that calls `floe_division_by_zero(line, col)` (marked `noreturn`) followed by `unreachable`.

3.4 Pipelines and lambdas

Semantic analysis has already turned `xs |> filter(x -> x > 3) |> map(x -> x * 2)` into `map(filter(xs, …), …)`. The code generator expands `filter` and `map` **inline**: a loop over the source array, with the

parameter `x` stored in an `alloca` and the lambda body emitted directly in the loop body. There are no closures and no function pointers. A lambda can therefore read any variable in scope at the call site, at zero cost.

3.5 The runtime library

Code that would be long or repetitive in IR is in `floe_runtime.c`, and the IR calls it:

Purpose	Functions
arrays	`floe_array_new`, `_len`, `_get` / `_set` (bounds-checked), `_push`, `_append`, `_slice`, `_reverse`, `_sort`, `_sum`, `_min`, `_max`, `floe_range`
strings	`floe_string_concat`, `_compare`, `_len`, `floe_int/float/bool_to_string`
printing	`floe_print_int`, `_float`, `_bool`, `_string`, `_array`, `_space`, `_newline`
errors	`floe_set_source_name`, `floe_runtime_error`, `floe_division_by_zero`

Each runtime function is declared in the module on first use (`Module::getOrInsertFunction`). The Phase 1 interpreter uses the same print functions, which is why both phases print byte-identical output.

3.6 From IR to an executable

`floec` initialises LLVM's native target, sets the module's target triple and data layout, and asks the `TargetMachine` to emit an object file (with position-independent code, as Ubuntu links PIE executables by default). It then runs `cc program.o build/floe_runtime.o -o program -lm` itself.

The IR can also be compiled by hand with the standard LLVM tools:

```
./build/floec --emit-llvm tests/mergesort.floe -o mergesort.ll
llc-16 -relocation-model=pic -filetype=obj mergesort.ll -o mergesort.o
cc mergesort.o build/floe_runtime.o -o mergesort -lm
./mergesort
# or, in one step:  clang-16 mergesort.ll build/floe_runtime.o -o mergesort -lm
```

---

4. How to check it

```
make compiler                                    # builds build/floec and build/floe_runtime.o
./build/floec tests/mergesort.floe -o mergesort   # compile to a native executable
./mergesort
./build/floec --emit-llvm tests/basic_pipeline.floe -o basic.ll   # look at the IR
./build/floec -O --emit-llvm tests/basic_pipeline.floe -o basic_opt.ll
make test-compiler                               # compile + run every test program
```

`make test-compiler` compiles every `tests/*.floe` and `tests/errors/runtime_*.floe` to a native executable, runs it, and checks its output, error message and exit status against the same expected files as the interpreter. Passing means **Phase 1 and Phase 2 agree on every program**. We also checked that all programs give the same output when compiled with `-O`, and that the compiler and the compiled programs are clean under AddressSanitizer and UndefinedBehaviorSanitizer.

5. Output

Compiling and running:

```
$ ./build/floec tests/mergesort.floe -o mergesort && ./mergesort
[1, 2, 3, 5, 8, 9]

$ file mergesort
mergesort: ELF 64-bit LSB pie executable, x86-64, ...
```

The IR for a small function (`fn max_of(a: int, b: int) -> int { if a > b { a } else { b } }`):

```
define internal i64 @fn.max_of(i64 %a, i64 %b) {
entry:
  %b2 = alloca i64, align 8
  %a1 = alloca i64, align 8
  store i64 %a, ptr %a1, align 8
  store i64 %b, ptr %b2, align 8
  %a3 = load i64, ptr %a1, align 8
  %b4 = load i64, ptr %b2, align 8
  %0 = icmp sgt i64 %a3, %b4
  br i1 %0, label %if.then, label %if.else

if.then:                                          ; preds = %entry
  %a5 = load i64, ptr %a1, align 8
  br label %if.end

if.else:                                          ; preds = %entry
  %b6 = load i64, ptr %b2, align 8
  br label %if.end

if.end:                                           ; preds = %if.else, %if.then
  %if.value = phi i64 [ %a5, %if.then ], [ %b6, %if.else ]
  ret i64 %if.value
}
```

A pipeline (`tests/basic_pipeline.floe`, `nums |> filter(x -> x > 3) |> map(x -> x * 2) |> print();`), excerpt:

```
@g.nums = internal global ptr null

filter.body:                                      ; preds = %filter.cond
  %4 = call i64 @floe_array_get(ptr %nums, i64 %i, i64 4, i64 8)
  store i64 %4, ptr %x, align 8
  %x1 = load i64, ptr %x, align 8
  %5 = icmp sgt i64 %x1, 3                        ; the lambda body, inlined
  br i1 %5, label %filter.keep, label %filter.next
...
map.body:                                         ; preds = %map.cond
  %11 = call i64 @floe_array_get(ptr %2, i64 %i2, i64 5, i64 8)
  store i64 %11, ptr %x3, align 8
  %x4 = load i64, ptr %x3, align 8
  %12 = mul i64 %x4, 2                            ; the lambda body, inlined
  call void @floe_array_set(ptr %9, i64 %i2, i64 %12, i64 5, i64 8)
  br label %map.next
...
map.end:                                          ; preds = %map.cond
  call void @floe_print_array(ptr %9, i32 0, i32 1)
  call void @floe_print_newline()
  ret i32 0
```

With `-O` LLVM folds `print(add(2, 3), max_of(4, 9))` completely:

```
define i32 @main() local_unnamed_addr {
entry:
  tail call void @floe_set_source_name(ptr nonnull @source.name)
  tail call void @floe_print_int(i64 5)
  tail call void @floe_print_space()
  tail call void @floe_print_int(i64 9)
  tail call void @floe_print_newline()
  ret i32 0
}
```

Runtime errors behave as in the interpreter:

```
$ ./build/floec tests/errors/runtime_division.floe -o div && ./div ; echo "exit status $?"
5
tests/errors/runtime_division.floe:1:37: runtime error: division by zero
exit status 1
```

Tests:

```
$ make test-compiler
PASS  compiler     tests/algorithms.floe
PASS  compiler     tests/basic_pipeline.floe
...
PASS  compiler     tests/errors/runtime_slice.floe

16 passed, 0 failed
```